

Estimating Website LCP Improvements Using PageSpeed Insights Diagnostics

Alexander Tran

ABSTRACT

Website performance affects user experience, but website owners may have difficulty determining which technical improvements are most likely to reduce page load delays. Google PageSpeed Insights (PSI) and Lighthouse provide performance measurements and diagnostic recommendations, but they do not directly estimate the combined Core Web Vitals (CWV) impact of changing several diagnostic factors. This project develops a machine-learning dashboard that estimates potential CWV changes after selected website performance metrics are adjusted. A Databricks data pipeline collected approximately 19,000 PSI observations from a broad set of websites. An XGBoost regression model was trained to estimate CWVs using diagnostic metrics with a log transformation applied to the target to reduce the influence of extreme LCP values. The final dashboard retrieves current PSI data from a user-provided URL. This allows users to adjust selected diagnostic metrics using reference-data percentiles and returns an estimate of the resulting changes. The model achieved a test mean absolute error of approximately 1,350 milliseconds. The completed system focuses on LCP and will extend to additional performance metrics and category-specific benchmarking.

1. INTRODUCTION

Website performance is an important part of the user experience. Pages that take a long time to load can make a website feel unresponsive and may discourage users from continuing with it. To help evaluate website performance, Google defines Core Web Vitals, a set of user-centered metrics that includes Largest Contentful Paint (LCP), Cumulative Layout Shift (CLS), and Interaction to Next Paint (INP). LCP measures how quickly the largest visible text or image element is rendered and is commonly used as an indicator of perceived loading speed.

Google PageSpeed Insights and Lighthouse provide useful performance measurements for individual webpages. These tools report LCP and other metrics and technical issues, such as unused JavaScript, unused CSS, large resource

payloads, render-blocking behavior, and slow server response times. Although this diagnostic information is valuable, users may still have difficulty deciding which changes to prioritize or estimating how several changes could affect overall loading performance.

This project attempts to address that gap by developing an interactive LCP estimation dashboard. Rather than attempting to replace the recommendations already provided by Lighthouse, the system uses PSI diagnostic data as input to a machine-learning model. After a user tests a website, the dashboard allows selected diagnostic and resource metrics to be adjusted to percentile-based values drawn from a reference dataset. The model then estimates the potential LCP outcome associated with those changes.

The project originally considered predicting multiple Core Web Vitals and generating a broad recommendation system. During development, the scope was refined after recognizing that Lighthouse already offers direct optimization recommendations. The final implementation focuses on a more specific and testable task. This scope allowed the project to demonstrate an end-to-end data-science workflow consisting of API-based data collection, feature engineering, regression modeling, model evaluation, and dashboard deployment.

2. RELATED WORK

Existing website performance tools like PSI and Lighthouse are widely used tools for measuring and diagnosing website performance. PSI reports page performance for both mobile and desktop strategies and combines laboratory audit data with available field data from Chrome users. Lighthouse provides laboratory audit data used by PSI and evaluates performance, best practices, and search engine optimization.

Existing tools are primarily designed to evaluate and diagnose an individual website. They identify issues and, in some cases, report estimated savings for an individual audit. However, they do not provide an interactive model that allows a user to adjust multiple diagnostic factors to estimate the resulting LCP outcome for a specific page. The purpose of this project is not to replace Lighthouse or PSI

recommendations. Instead, it uses PSI diagnostics as inputs to a machine learning model that supports post-test analysis. Gradient-boosted decision trees are well suited to structured tabular data because they can model nonlinear relationships and interactions within variables. In this project, XGBoost was selected to estimate LCP from PageSpeed Insight performance outputs. The current implementation focuses on LCP as the first completed Core Web Vitals component. The broader framework is intended to support future models for LCP and INP, along with supporting metrics such as FCP and Time to Interactive (TTI).

3. PROPOSED WORK

This section describes the final implemented system. The project consists of a Databricks data-collection and feature-engineering pipeline, an XGBoost regression model for LCP estimation, and a Streamlit dashboard that allows users to explore model-based optimizations.

3.1 Data Collection Pipeline

The dataset was collected through a repeatable Databricks pipeline. The process begins with a large set list of websites. A collection script processes each website to identify relevant subdomains for testing. Testing subdomains is important because different subdomains can represent distinct website sections, applications, or content experiences with different performance characteristics.

A second script queried the PSI API for each collected URL. Each URL was tested using both the mobile and desktop strategies. The API returned raw JSON responses, diagnostic opportunities, and available field metrics. Collecting both strategies allowed the dataset to preserve the distinction between mobile and desktop performance measurements.

The raw API responses were then processed by a feature-extraction script. This script converted nested JSON objects into structured rows for analysis and machine learning models. Approximately 19,000 observations were collected. Some websites returned incomplete or unusable lab data, so the modeling dataset was limited to about 16,000 rows.

3.1 Feature Engineering and Dataset Design

The feature-extraction process retained identifying information for each observation, including the domain, tested URL, device strategy, and PSI performance score.

The primary target variable was LCP. LCP was selected because it is a CWV that represents the loading time of the largest visible content element on a page. The dataset also retrained other performance metrics, including LCP, FCP, TBT, TTI, and speed index. These variables support exploratory analysis and provide a foundation for future CWV models.

Other diagnostic features include total byte weight, unused CSS, unused JavaScript, unminified CSS, unminified JavaScript, network server latency, image-resource bytes, front-resource bytes, potential JavaScript savings, and many more. The pipeline also extracted field metrics, including INP and experimental TFB when available.

Feature Groups	Examples
Identification and context	Domain, URL, mobile or desktop strategy, performance score
Core performance metrics	LCP, CLS, FCP, TBT, Speed Index, TTI
Resource and diagnostic metrics	Total byte weight, unused JavaScript and CSS, Image bytes, font bytes, etc.
Field metrics	INP and TFB

Table 1. Data separation and included groups

The dashboard uses a selected subset of these diagnostics and resource metrics as adjustable inputs. The mobile and desktop reference data are stored separately so that user-selected percentiles are based on the relevant device strategy.

3.3 The current implementation uses an XGBoost regression model that estimates LCP. Because the LCP distribution contained extremely high values, the target was transformed with log1p during model training and converted back to milliseconds after prediction. This approach reduces the influence of extreme outliers while preserving the original LCP.

The model was evaluated using five-fold cross-validation on the training data and a separate test set. Main Absolute Error (MAE) was selected as the primary evaluation metric because it expresses average prediction error and is less dominated by extreme LCP values than Root Mean Squared Error (RMSE). RMSE and R-squared were retrained as supporting metrics to assess larger errors and overall variation explained by the model.

3.4 Dashboard Estimation

The Streamlit dashboard begins when a user enters a website URL and selects either mobile or desktop testing. The dashboard retrieves the current PSI results and displays the website’s observed LCP. It then calculates the current percentile of each adjustable metric within the matching mobile or desktop reference dataset.

Users can move percentile sliders to simulate lower or higher values for selected diagnostic metrics. For example, a user may set unused JavaScript to a lower percentile, representing a scenario where the website has less of that resource burden than its current result. The selected percentile is converted into a corresponding value from the reference dataset, and the modified feature values are passed to the trained model.

The dashboard predicts both a baseline model value and a modified model value. Rather than displaying the raw modified prediction directly, it applies the relative model change to the website’s actual PSI-measured LCP:

$$\widehat{LCP}_{new} = \widehat{LCP}_{current} \times \frac{\widehat{LCP}_{modified}}{\widehat{LCP}_{baseline}}$$

This calculation preserves the user’s observed PSI measurement while using the model to estimate the relative effect of the simulated changes. The output is presented as an estimated new LCP and as an estimated percentage improvement.

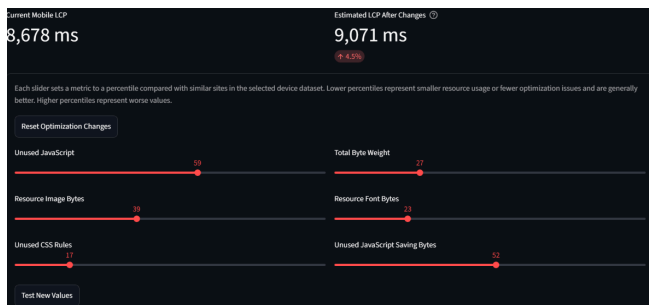


Figure 1. Streamlit dashboard displaying current LCP, adjustable percentile diagnostic inputs, and the model-estimated LCP after simulated optimization.

4. EVALUATION

4.1 Evaluation Approach

The LCP prediction model was evaluated using both five-fold cross-validation (CV) on the training data and a separate test set. Cross-validation was used to estimate how consistently the model performed across different subsets of the training data. The test set was not used during model fitting to provide a separate estimation of performance.

MAE was selected as the primary metric because it reports the average difference between predicted and actual LCP values. This makes it directly interpretable for performance data. RMSE was calculated because it gives greater weight to larger errors.

4.2 Model Performance Results

Table 2 summarizes the LCP model results. The mean five-fold cross-validation MAE was 1,51 milliseconds, with a standard deviation of approximately 85 milliseconds across the folds. The test MAE was 1,450 milliseconds.

Metric	Result
Mean 5-fold CV MAE	1,541 ms
CV MAE standard deviation	85 ms
Test ME	1,350 ms
Test RMSSE	3,378 ms
R-squared	0.861

Table 2. LCP prediction evaluation results.

The test R-squared of 0.861 suggests that the model explained a substantial portion of the variation in LCP. However, RMSE was much higher than MAE. This difference suggests that a smaller number of observations had larger prediction errors. This pattern is expected because website LCP data contains a long right trail of unusually slow pages.

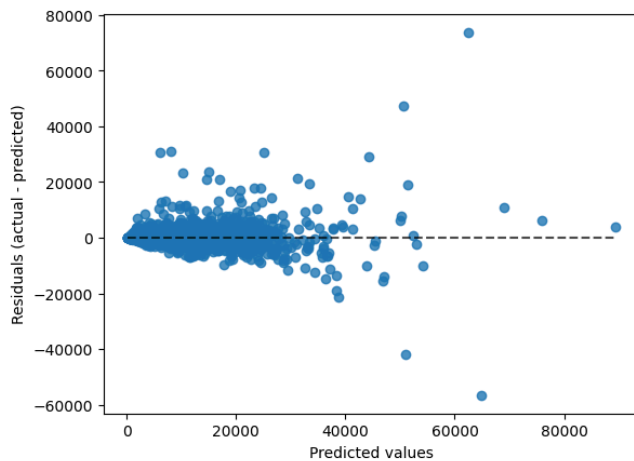


Figure 2. Residuals versus predicted LCP values on the test set.

Figure 2 shows that residuals were generally centered near zero for most observations. However, residual variability increased as predicted LCP became larger. This pattern supports the use of MAE as the primary performance metric.

The model evaluation measures the accuracy of LCP prediction. It does not directly validate whether every dashboard scenario will produce the same change after a user modifies a live website.

5. Discussion

5.1 Interpretation of the LCP model

The evaluation results indicate that PSI diagnostic metrics contain useful predictive information about LCP. The model achieved an average error of 1.35 seconds while explaining about 86% of the variation in the test LCP values.

Feature-importance analysis showed that device strategy and unused JavaScript were among the more influential predictors in the model. This result is consistent with the fact that mobile and desktop PSI results operate under different conditions. Feature importance should not be treated as proof that changing one variable will cause a specific LCP improvement. The dashboard estimates a possible LCP scenario based on patterns in the collected data rather than real outcomes.

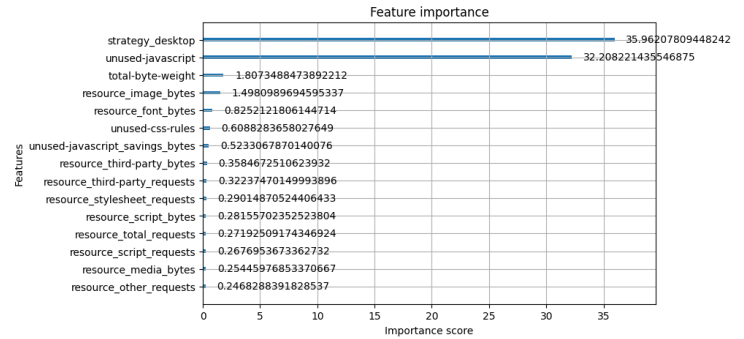


Figure 3. Feature importance values from the LCP model. Higher values indicate greater contribution to the predictions.

5.2 Limitations

The primary limitation is the scope. Although the project is designed as a broader CWV framework, the current dashboard implements a model only for LCP. CLS and INP, the other CWVs are not yet available as dashboard prediction outputs. Supporting metrics such as FCP and TTI are retained in the dataset but are also not currently modeled.

A second limitation is that PSI insights lab results are controlled measurements rather than real user experience. Results can vary due to testing conditions, network behaviors, device, server, and website changes.

The dashboard also uses a selected set of diagnostic metrics as adjustable inputs. In a real optimization effort, changing one resource metric may affect other metrics at the same time. The percentile sliders simplify this relationship so users can explore scenarios.

5.3 Future Work

Future work will include metrics beyond LCP. The next models will focus on CLS and INP. Addition models for FCP, TTI, and TBT could provide users with a broader view of performance.

A second planned improvement is category-based benchmarking. Rather than comparing a website only with all sites in the entire dataset, the dashboard could compare it with websites in the same category and device. For example, a user could see whether a mobile e-commerce website has an LCP that is lower than most other mobile e-commerce websites.

6. CONCLUSION

This project creates a machine learning dashboard designed to estimate how optimization changes may affect Core Web Vitals. The completed implementation focuses on Largest Contentful Paint.

A Databricks pipeline collected PageSpeed Insights data from a large set of websites. It then processes mobile and desktop results and converts raw API responses into structured performance features. An XGBoost regression model was created to estimate LCP. The model achieved a test MAE of about 1,350 milliseconds and an R-squared of 0.861.

In the Streamlit dashboard, users can retrieve a website's current PageSpeed Insight result, adjust selected diagnostic metrics, and receive an estimate of the potential LCP outcome. The dashboard is intended to support optimization decisions, not replace an actual PageSpeed Insights test.

Overall, the project demonstrates an end-to-end data science workflow that combines API data collection, feature engineering, regression modeling, evaluation, and interactive deployment. Future work will extend the system to Cumulative Layout Shift, Interaction to Next Paint, First Contentful Paint, and Time to Interactive. There will also be category benchmarking.

REFERENCES

Philip Walton. 2024. Web Vitals. web.dev.
<https://web.dev/articles/vitals>

Google. 2024. About PageSpeed Insights. Google for Developers.
<https://developers.google.com/speed/docs/insights/v5/about>

Google. n.d. Lighthouse Overview. Chrome for Developers.
<https://developer.chrome.com/docs/lighthouse/overview/>